

Lecture 06

Object-Oriented Programming

{OOP}

Overriding, Overloading

Content

- `Overriding/overloading`
- `Introduction to Swing`

Overriding Method

If subclass (child class) has the *same method as declared in the parent class*, it is known as method overriding in Java

Rules for Java Method Overriding:

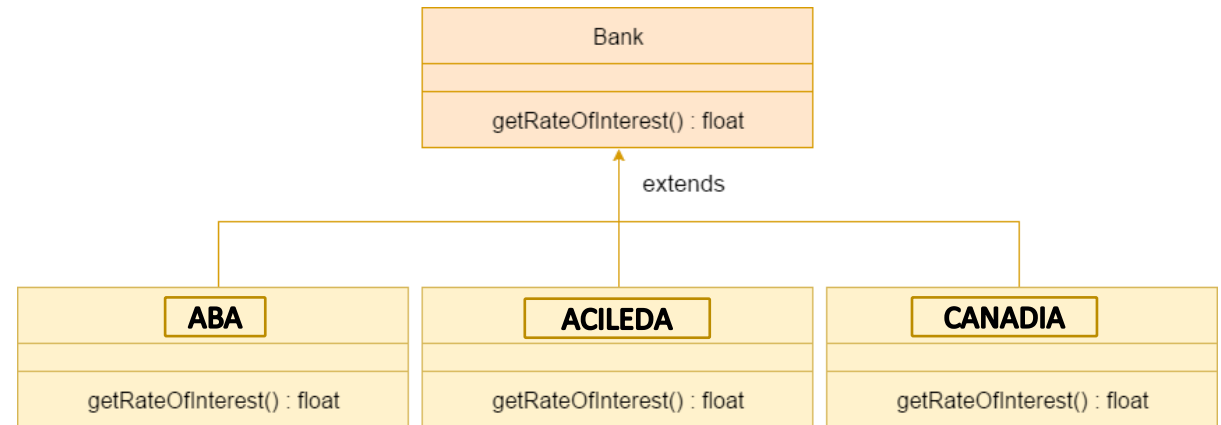
- ✓ The method must have the same name as in the parent class
- ✓ The method must have the same parameter as in the parent class.
- ✓ There must be inheritance relationship .

```
1 class Vehicle {  
2     void run() {  
3         System.out.println("Vehicle is running");  
4     }  
5 }  
6  
7 class Bike extends Vehicle {  
8     void run() {  
9         System.out.println("Bike is running safely");  
10    }  
11 }  
12  
13 public class Simple {  
14     public static void main(String args[]) {  
15         Bike obj = new Bike();  
16         obj.run();  
17     }  
18 }
```

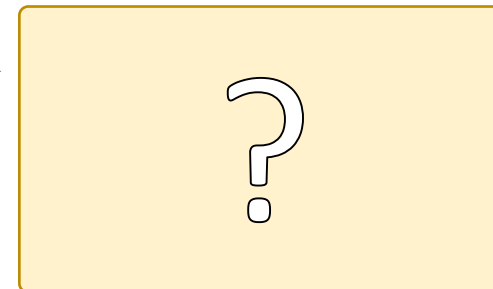
Overriding Method

Example

```
1 class Bank {  
2     int getRateOfInterest() {  
3         return 0;  
4     }  
5 }  
6  
7 class ABA extends Bank {  
8     int getRateOfInterest() {  
9         return 8;  
10    }  
11 }  
12  
13 class ACILEDA extends Bank {  
14     int getRateOfInterest() {  
15         return 7;  
16    }  
17 }  
18 class CANADIA extends Bank {  
19     int getRateOfInterest() {  
20         return 9;  
21    }  
22 }  
23  
24 class MainTest {  
25     public static void main(String args[]) {  
26         ABA aba = new ABA();  
27         ACILEDA aci = new ACILEDA();  
28         CANADIA cana = new CANADIA();  
29         System.out.println("ABA Rate of Interest: " + aba.getRateOfInterest());  
30         System.out.println("ACILEDA Rate of Interest: " + aci.getRateOfInterest());  
31         System.out.println("CANADIA Rate of Interest: " + cana.getRateOfInterest());  
32     }  
33 }
```



Output



Overloading Method

If a class has multiple methods *having same name but different in parameters*, it is known as Method Overloading.

👍 Method overloading increases the readability of the program.

Different ways to overload the method

1. By changing number of arguments/parameters
2. By changing the data type

❑ Method Overloading: By changing number of arguments

Simple.java

```
1 class Adder {
2     int add(int a,int b){
3         return a+b;
4     }
5     int add(int a,int b,int c){
6         return a+b+c;
7     }
8 }
9 class Simple {
10     public static void main(String[] args) {
11         Adder a = new Adder();
12         System.out.println(a.add(11,11));
13         System.out.println(a.add(11,11,11));
14     }
15 }
```

Simple.java (static access identifier)

```
1 class Adder {
2     static int add(int a,int b) {
3         return a+b;
4     }
5     static int add(int a,int b,int c) {
6         return a+b+c;
7     }
8 }
9 class Simple {
10     public static void main(String[] args) {
11         System.out.println(Adder.add(11,11));
12         System.out.println(Adder.add(11,11,11));
13     }
14 }
```

Overloading Method

❑ Method Overloading: changing data type of arguments

Simple.java

```
1 class Adder {
2     static int add(int a, int b) {
3         return a+b;
4     }
5     static double add(double a, double b) {
6         return a+b;
7     }
8 }
9 class Simple {
10    public static void main(String[] args) {
11        System.out.println(Adder.add(11, 11));
12        System.out.println(Adder.add(12.3, 12.6));
13    }
14 }
15 }
16 }
```

 Method overloading is **not possible** by **changing the return type of the method only** because of ambiguity

Simple.java

```
1 class Adder {
2     static int add(int a,int b) {
3         return a+b;
4     }
5     static double add(int a,int b) {
6         return a+b;
7     }
8 }
9 class Simple {
10    public static void main(String[] args) {
11        System.out.println(Adder.add(11, 11)); //ambiguity
12    }
13 }
```



Output:

Compile Time Error: method add(int,int) is already defined in class Adder

Overloading Method

Method Overloading: **main()** method

Simple.java

```
1 class Simple {
2     public static void main(String[] args) {
3         System.out.println("main with String[]");
4     }
5     public static void main(String args) {
6         System.out.println("main with String");
7     }
8     public static void main() {
9         System.out.println("main without args");
10    }
11 }
```

Output:



Method Overloading: **Type Promotion**

Math.java

```
1 class Math {
2     void sum(int a, long b) {
3         System.out.println(a+b);
4     }
5     void sum(int a, int b, int c) {
6         System.out.println(a+b+c);
7     }
8
9     public static void main(String args[]) {
10        Math obj = new Math();
11
12        obj.sum(20, 20); //now second int literal will be promoted to long
13        obj.sum(20, 20, 20);
14    }
15 }
```

Math.java

```
1 class Math {
2     void sum(int a, int b) {
3         System.out.println("int arg method invoked");
4     }
5     void sum(long a, long b) {
6         System.out.println("long arg method invoked");
7     }
8
9     public static void main(String args[]) {
10        Math obj = new Math();
11        obj.sum(20, 20); //now int arg sum() method gets invoked
12    }
13 }
```

Math.java

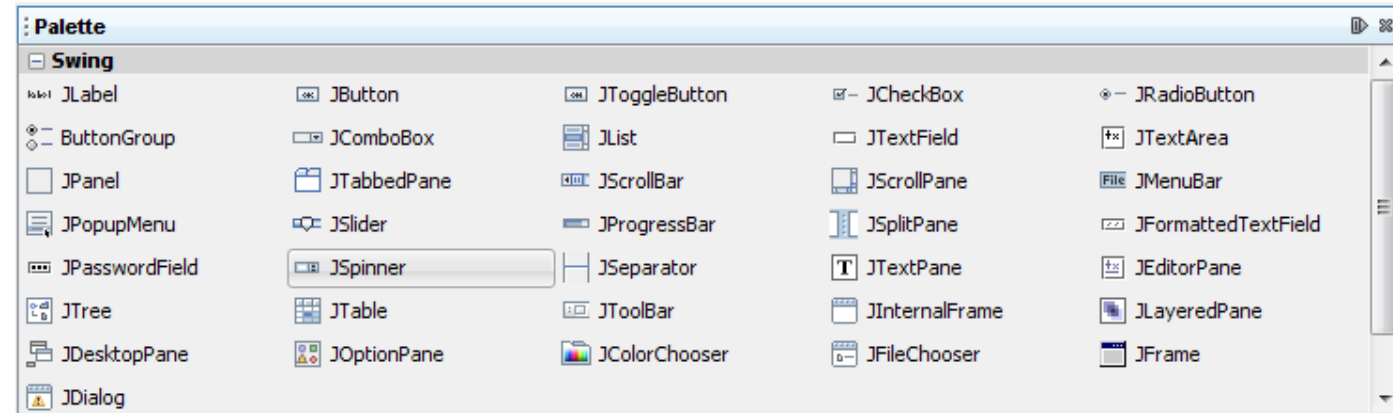
```
1 class Math {
2     void sum(int a, long b) {
3         System.out.println("a method invoked");
4     }
5     void sum(long a, int b) {
6         System.out.println("b method invoked");
7     }
8
9     public static void main(String args[]) {
10        Math obj = new Math();
11        obj.sum(20, 20); //now ambiguity
12    }
13 }
```



Introduction to Swing UI

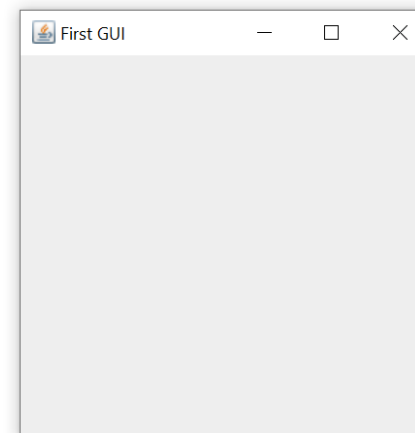
❏ What is Java Swing?

- Part of the Java Foundation Classes (JFC)
- Provides a **rich set of GUI components**
- Used to create a Java program with a **graphical user interface** (GUI)
- table controls, list controls, tree controls, buttons, and labels, and so on...



```
1 import javax.swing.*;
2
3 public class Test {
4     public static void main(String[] args) {
5         //creating instance of JFrame
6         JFrame f= new JFrame("First GUI");
7
8
9         f.setSize(300,300); // set frame size
10        f.setLayout(null); // ignore layout
11        f.setVisible(true); // set frame to be visible on window
12    }
13
14 }
15
```

Output :



Introduction to Swing UI

☐ Commonly used Methods of Component class

Method	Description
public void add(Component c)	add a component on another component.
public void setSize(int width,int height)	sets size of the component.
public void setLayout(LayoutManager m)	sets the layout manager for the component.
public void setVisible(boolean b)	sets the visibility of the component. It is by default false.

☐ setBounds()

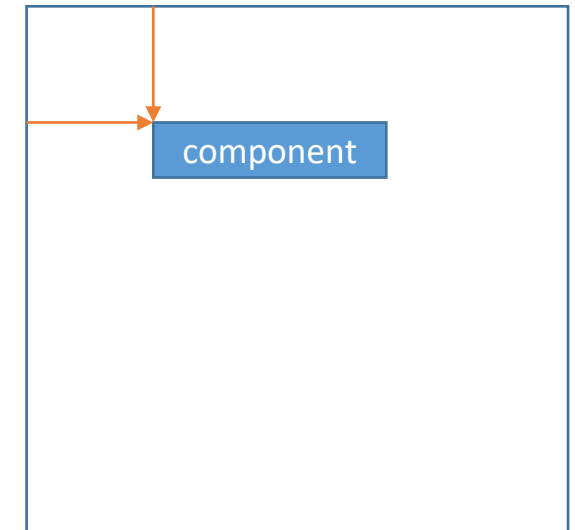
The **layout managers** are used to automatically decide the position and size of the added components

The setBounds() method needs four arguments:

- The first two arguments are **x and y coordinates** of the top-left corner of the component
- the third argument is the **width** of the component
- the fourth argument is the **height** of the component.

Syntax

```
setBounds(int x-coordinate, int y-coordinate, int width, int height)
```



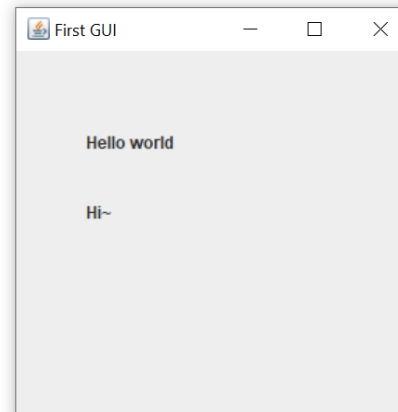
Introduction to Swing UI

❑ Labels

The object of **JLabel** class is a component for placing text in a container

```
1 import javax.swing.*;
2
3 public class Test {
4     public static void main(String[] args) {
5
6         // create a main frame
7         JFrame f= new JFrame("First GUI");
8
9         JLabel l1,l2;
10
11         l1=new JLabel("Hello world");
12         l1.setBounds(50,50, 100,30);
13         f.add(l1); // add Label 1 to frame
14
15         l2=new JLabel("Hi~");
16         l2.setBounds(50,100, 100,30);
17         f.add(l2); // add Label 2 to frame
18
19         f.setSize(300,300);
20         f.setLayout(null);
21         f.setVisible(true);
22     }
23 }
24 }
```

Output:



Object properties:

Methods	Description
String getText()	t returns the text string that a label displays.
void setText(String text)	It defines the single line of text this component will display.
void setHorizontalAlignment(int alignment)	It sets the alignment of the label's contents along the X axis.
Icon getIcon()	It returns the graphic image that the label displays.
int getHorizontalAlignment()	It returns the alignment of the label's contents along the X axis.

Introduction to Swing UI

❏ JButton

The **JButton** class is used to create a labeled button that is interactable by clicking

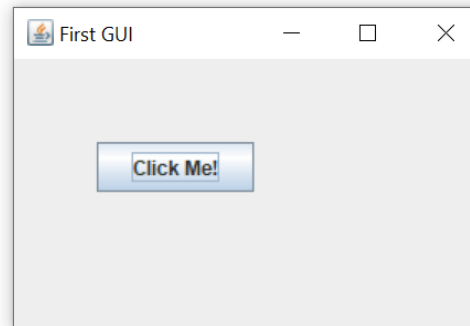
```
import javax.swing.*;

public class Test {
    public static void main(String[] args) {
        JFrame f=new JFrame("First GUI");

        JButton b=new JButton("Click Me!");
        b.setBounds(50,50,95,30);
        f.add(b);

        f.setSize(300,200);
        f.setLayout(null);
        f.setVisible(true);
    }
}
```

Output:



Object properties:

Methods	Description
void setText(String s)	It is used to set specified text on button
String getText()	It is used to return the text of the button.
void setEnabled(boolean b)	It is used to enable or disable the button.
void setIcon(Icon b)	It is used to set the specified Icon on the button.
Icon getIcon()	It is used to get the Icon of the button.
void setMnemonic(int a)	It is used to set the mnemonic on the button.
void addActionListener(ActionListener a)	It is used to add the action listener to this object.

Introduction to Swing UI

❑ JTextField

The **JTextField** class is a text component that allows the editing of a single line text

```
import javax.swing.*;
public class Test {
    public static void main(String[] args) {
        JFrame f = new JFrame("First GUI");

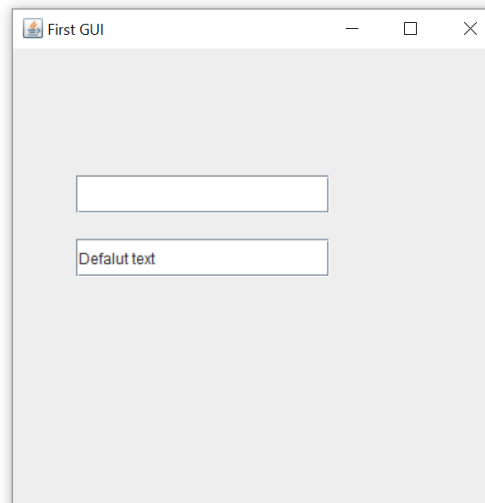
        JTextField t1, t2;
        t1 = new JTextField("");
        t1.setBounds(50, 100, 200, 30);

        t2 = new JTextField("Defalut text");
        t2.setBounds(50, 150, 200, 30);

        f.add(t1);
        f.add(t2);

        f.setSize(400, 400);
        f.setLayout(null);
        f.setVisible(true);
    }
}
```

Output:



Object properties:

Methods	Description
void addActionListener(ActionListener l)	It is used to add the specified action listener to receive action events from this textfield.
Action getAction()	It returns the currently set Action for this ActionEvent source, or null if no Action is set.
void setFont(Font f)	It is used to set the current font.
void removeActionListener(ActionListener l)	It is used to remove the specified action listener so that it no longer receives action events from this textfield.

Introduction to Swing UI

❑ Event Handling: Click

- Event Handling is the mechanism that controls the event and decides what should happen if an event occurs
- This mechanism has a code which is known as an event handler, that is executed when an event occurs

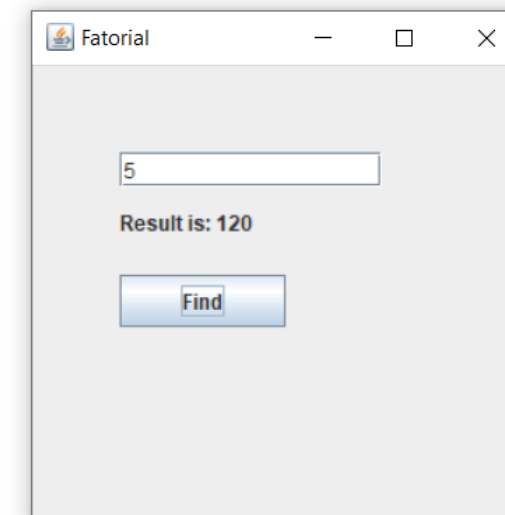
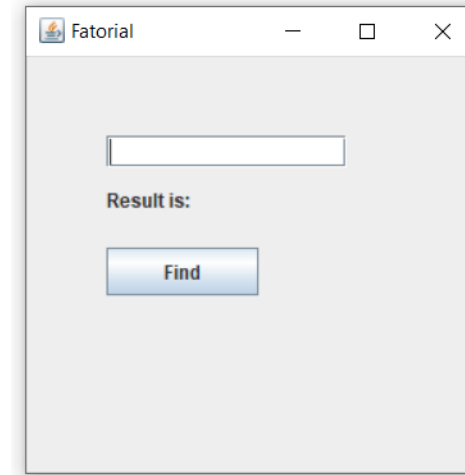
```
import java.awt.event.*;   
import javax.swing.*;
```

```
public class Test {  
    public static void main(String[] args) {  
        JFrame f = new JFrame("Fatorial");  
  
        JButton b = new JButton("Click Me");  
        b.setBounds(50, 120, 95, 30);  
        f.add(b);  
  
        b.addActionListener(new ActionListener() {   
            public void actionPerformed(ActionEvent e) {   
                // to do something   
            }  
        });  
  
        f.setSize(300, 300);  
        f.setLayout(null);  
        f.setVisible(true);  
    }  
}
```

Introduction to Swing UI

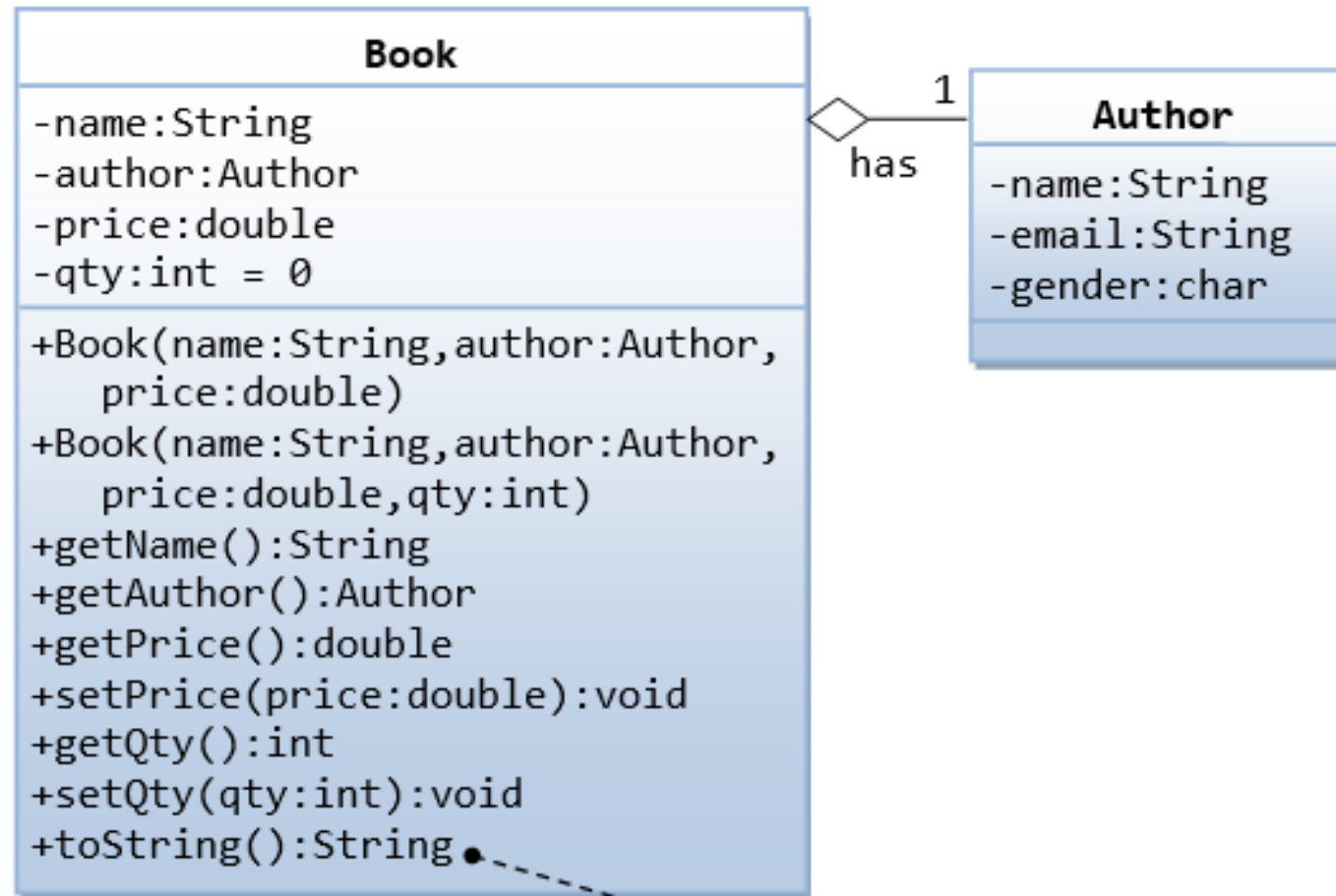
□ Event Handling: Example

```
public class Test {  
    public static void main(String[] args) {  
        JFrame f = new JFrame("Fatorial");  
        final JTextField tf = new JTextField();  
        tf.setBounds(50, 50, 150, 20);  
        f.add(tf);  
  
        JLabel l = new JLabel("Result is: ");  
        l.setBounds(50, 80, 150, 20);  
        f.add(l);  
  
        JButton b = new JButton("Find");  
        b.setBounds(50, 120, 95, 30);  
        b.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
  
                // Calculate factorial  
                int i, fact = 1;  
                int number = Integer.parseInt(tf.getText());  
                for (i = 1; i <= number; i++) {  
                    fact = fact * i;  
                }  
  
                l.setText("Result is: " + fact);  
            }  
        });  
        f.add(b);  
  
        f.setSize(300, 300);  
        f.setLayout(null);  
        f.setVisible(true);  
    }  
}
```



In-class practice

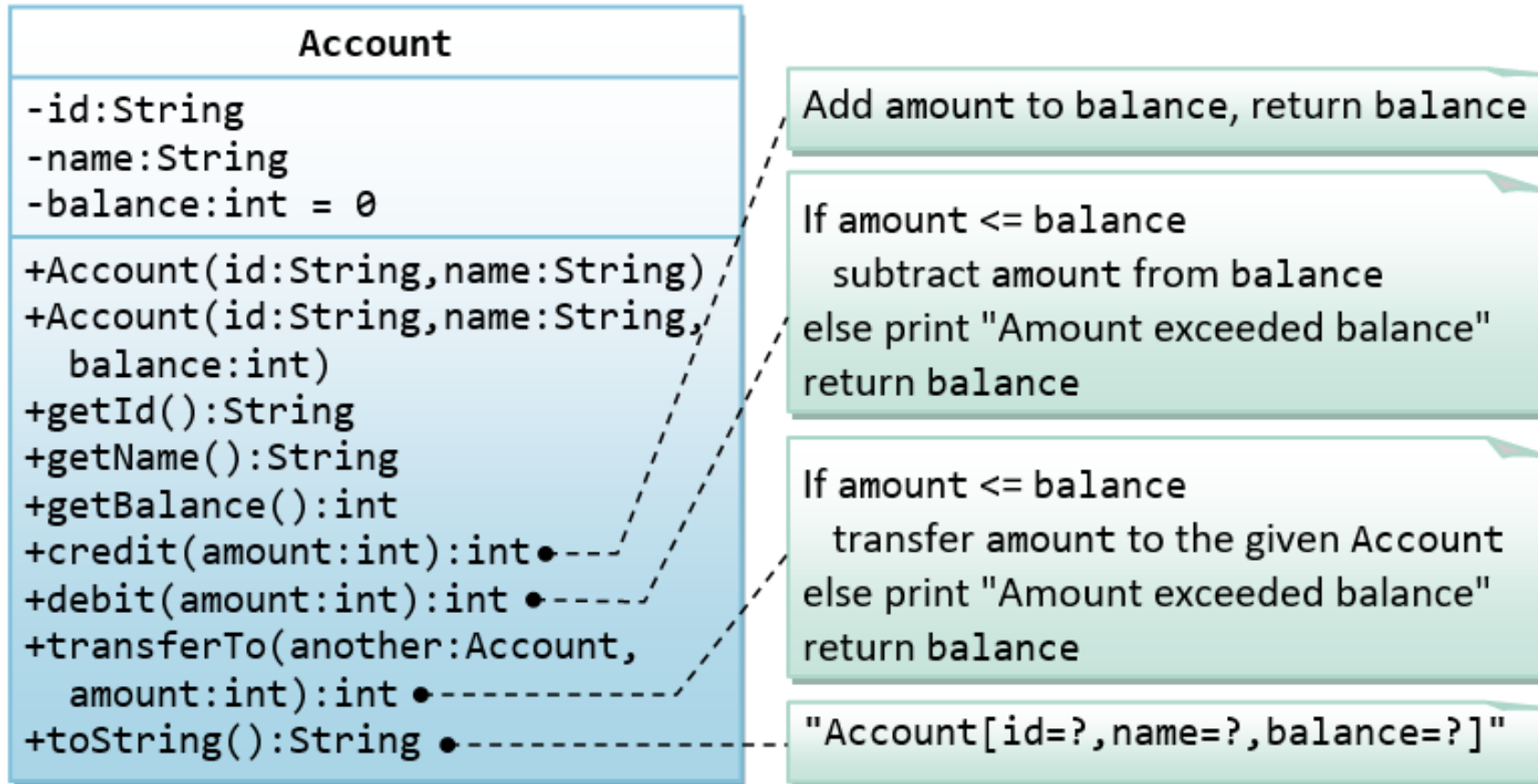
- Implement following class diagram in Java. Also, write a test to use the implemented class.



"Book[name=?, Author[name=?, email=?, gender=?], price=?, qty=?]"
You need to reuse Author's toString().

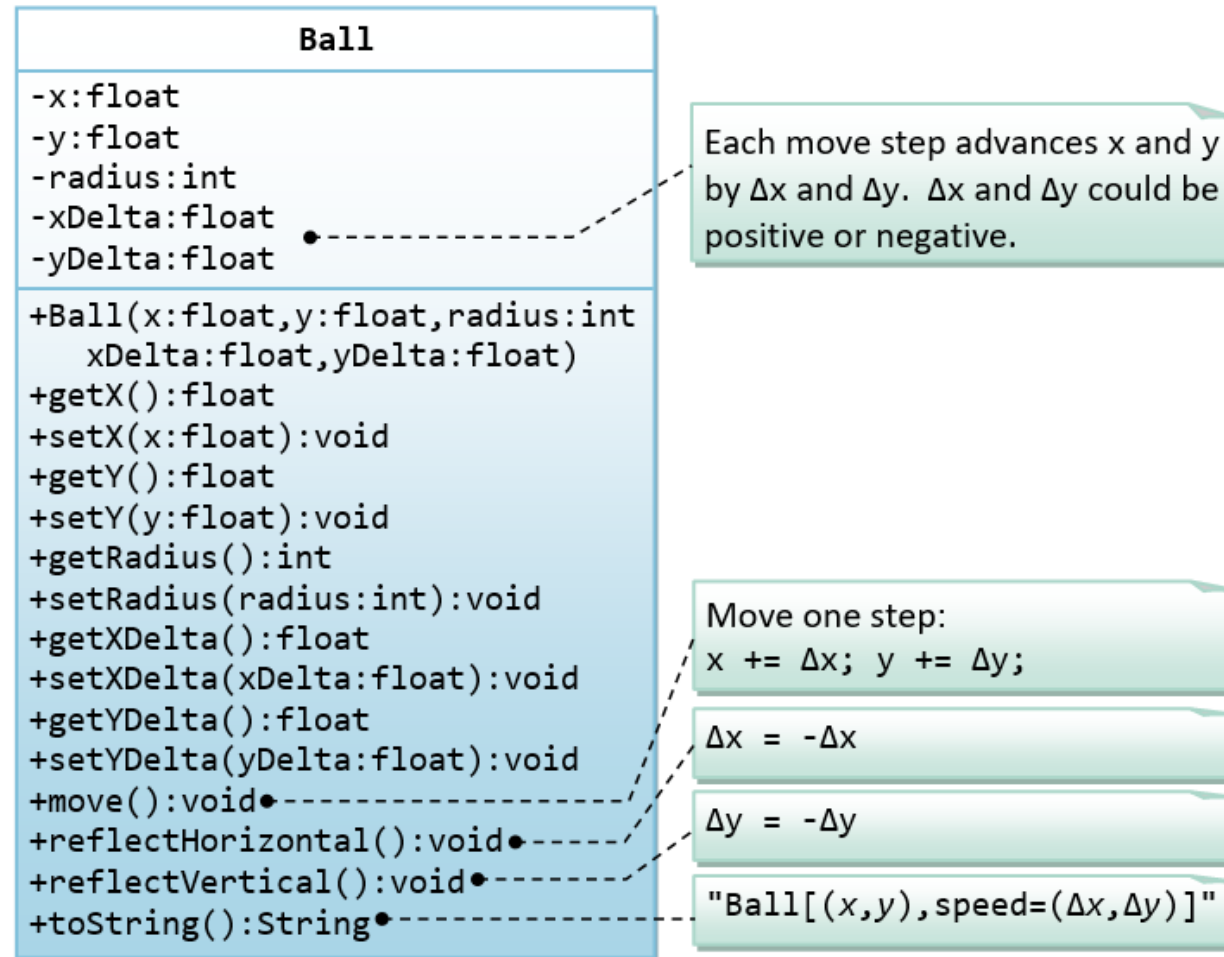
In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



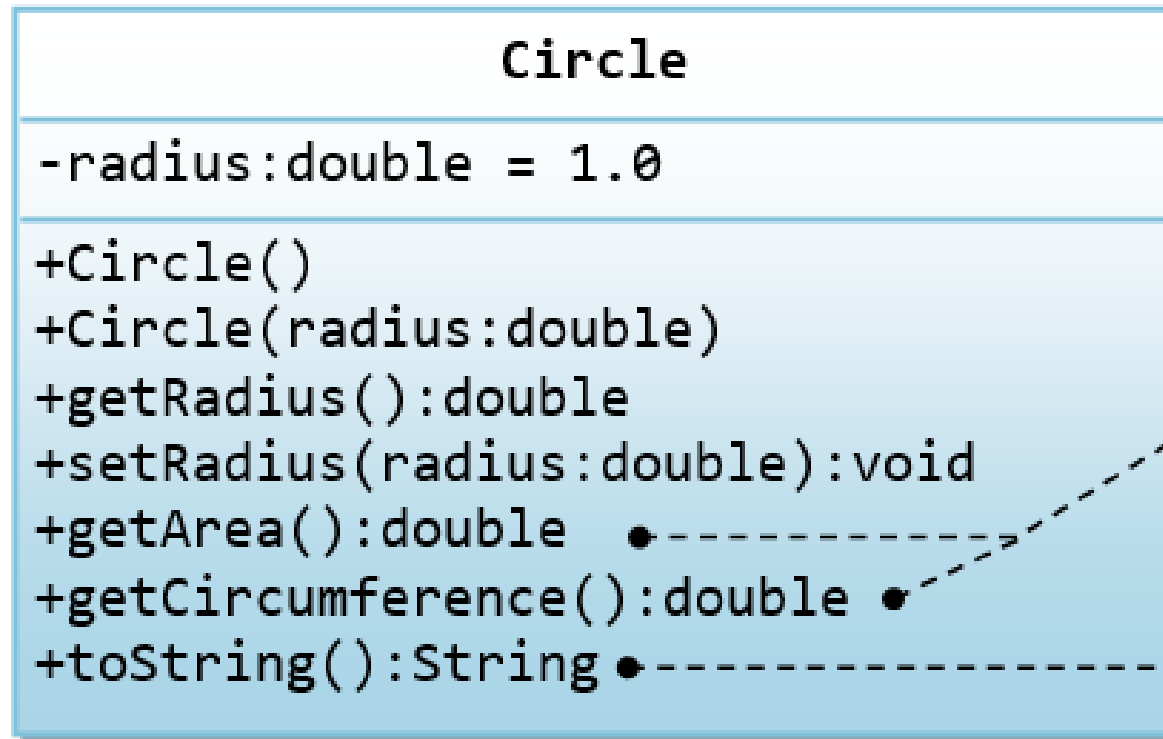
In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.

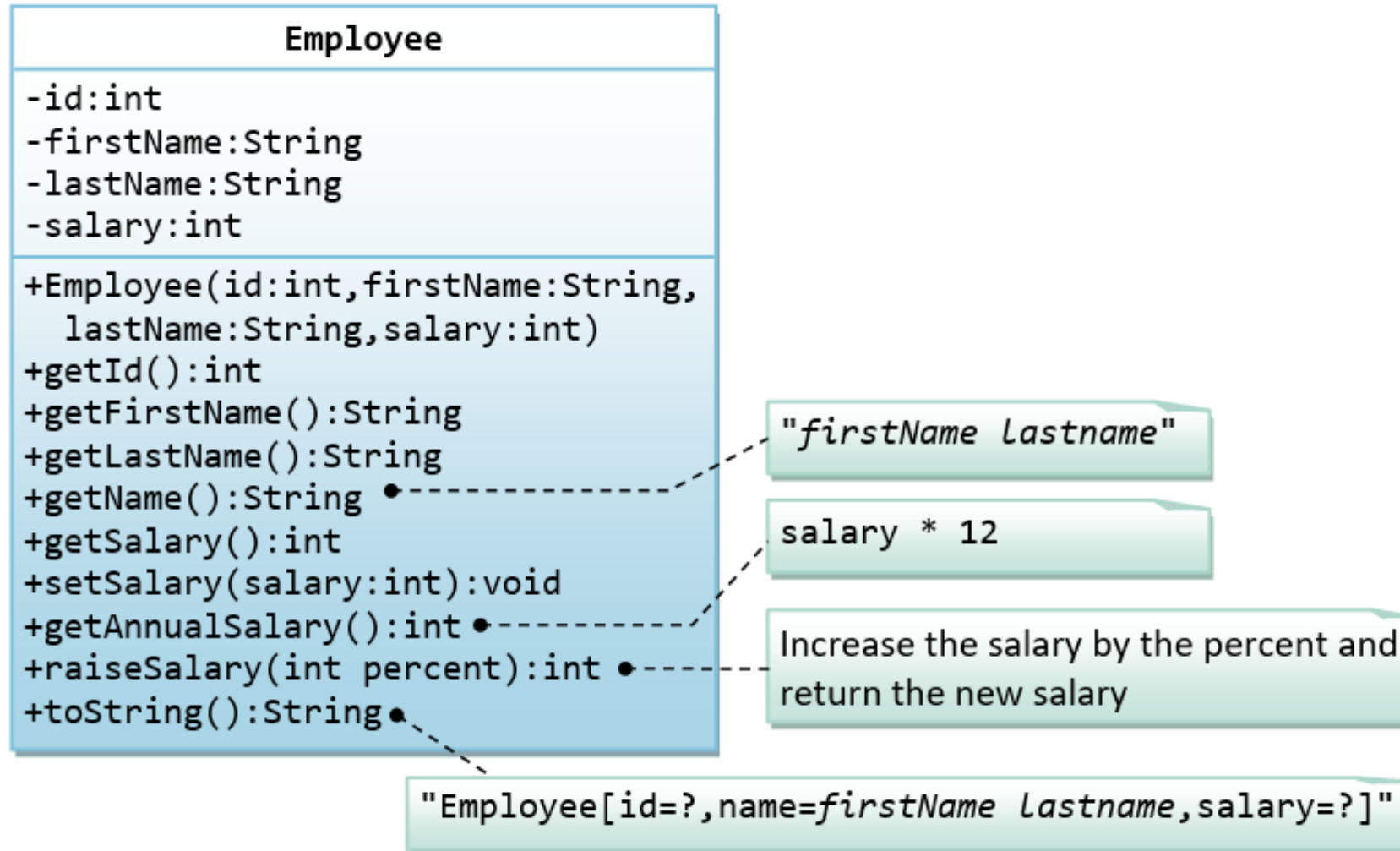


Use JDK constant `Math.PI` for π

"Circle[radius=?]"

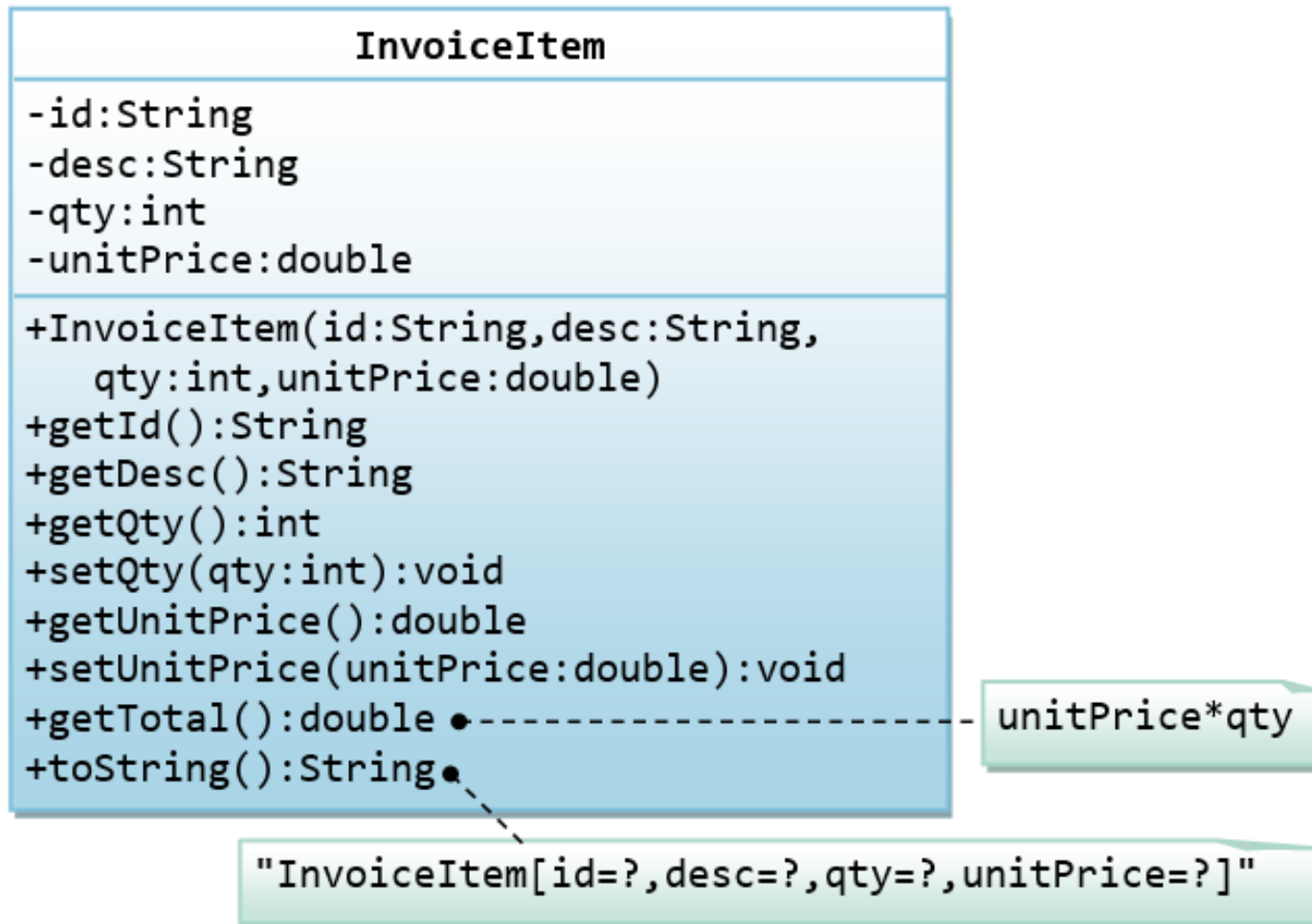
In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



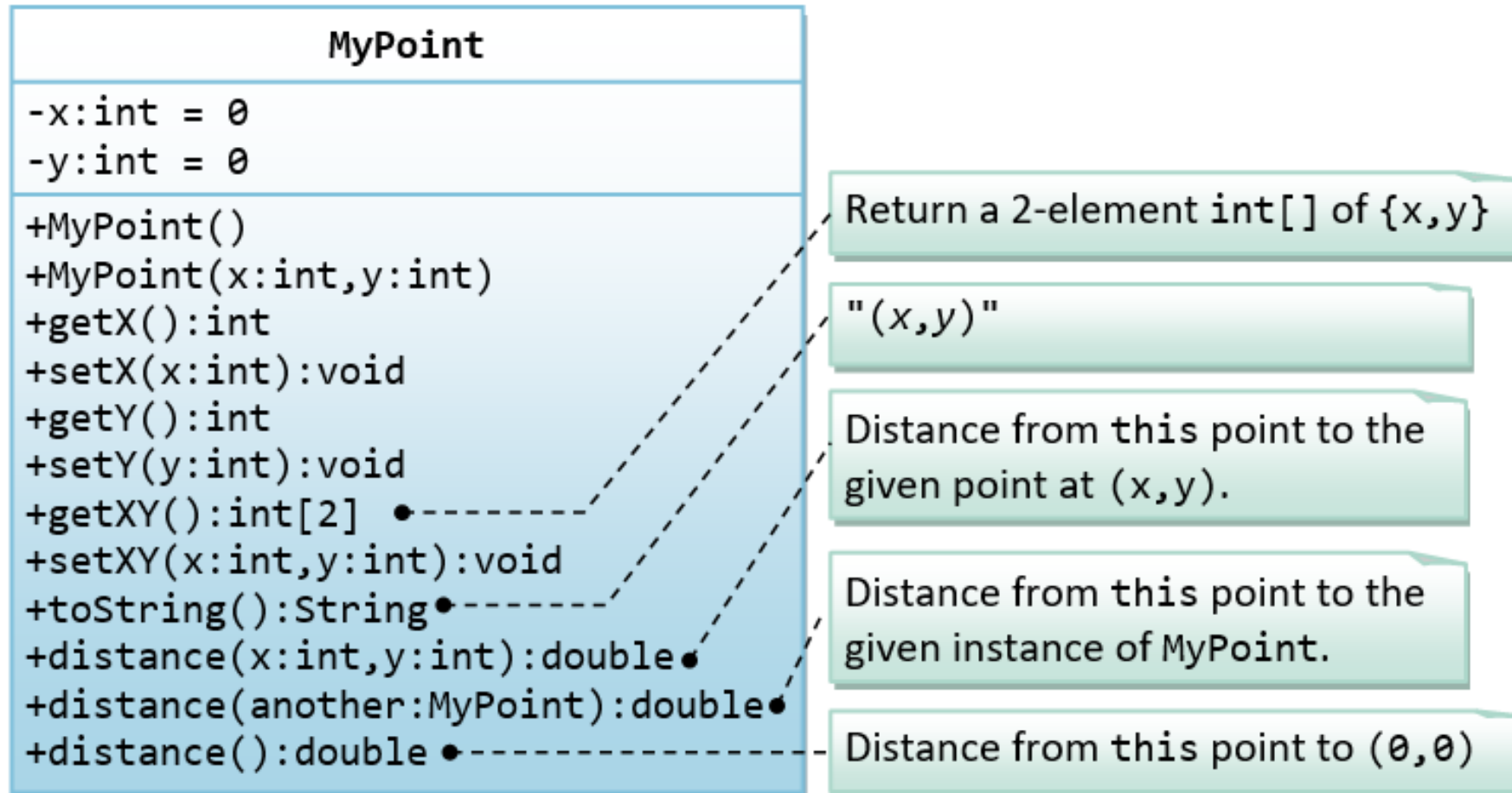
In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



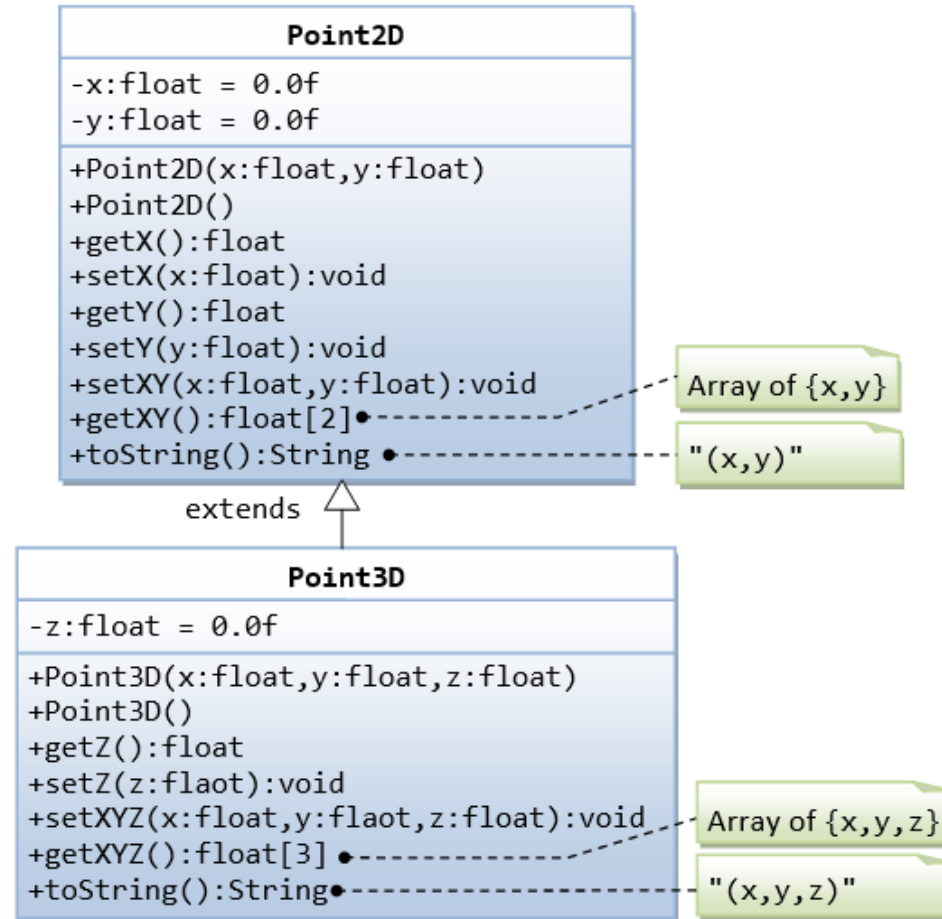
In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



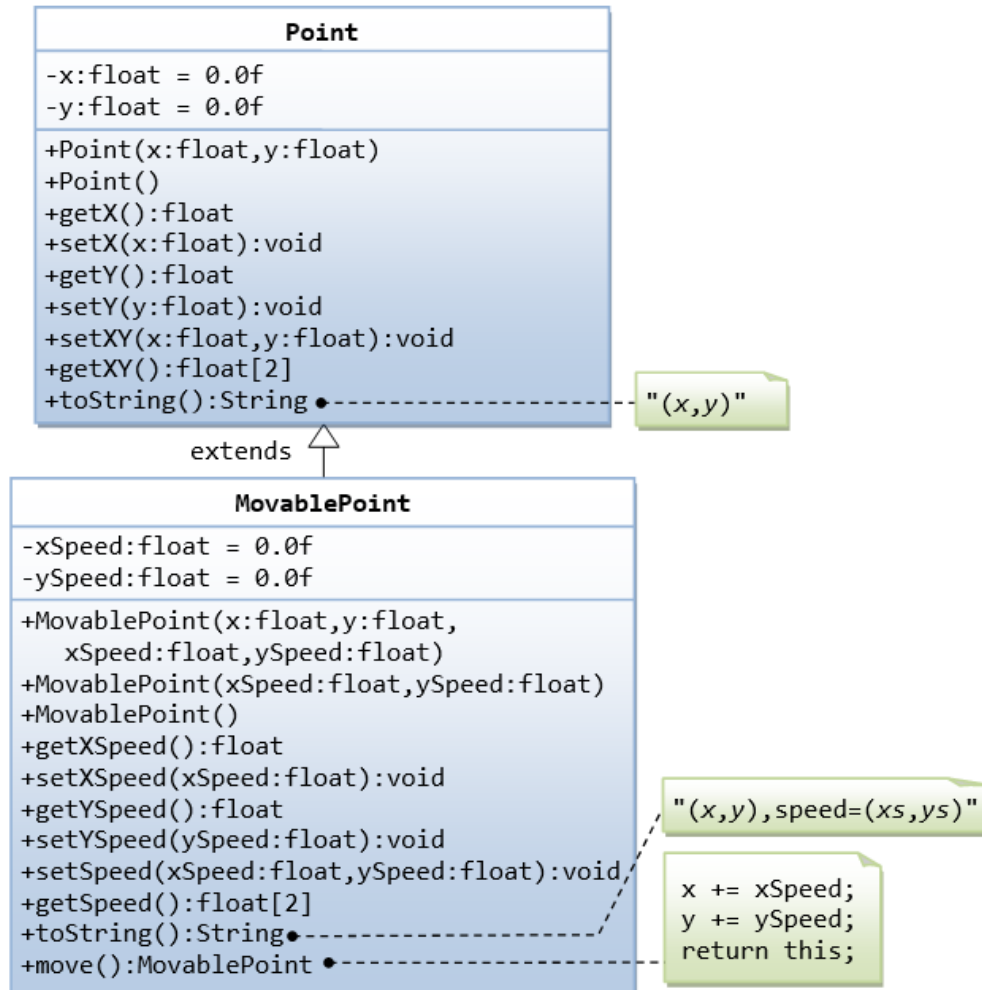
In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



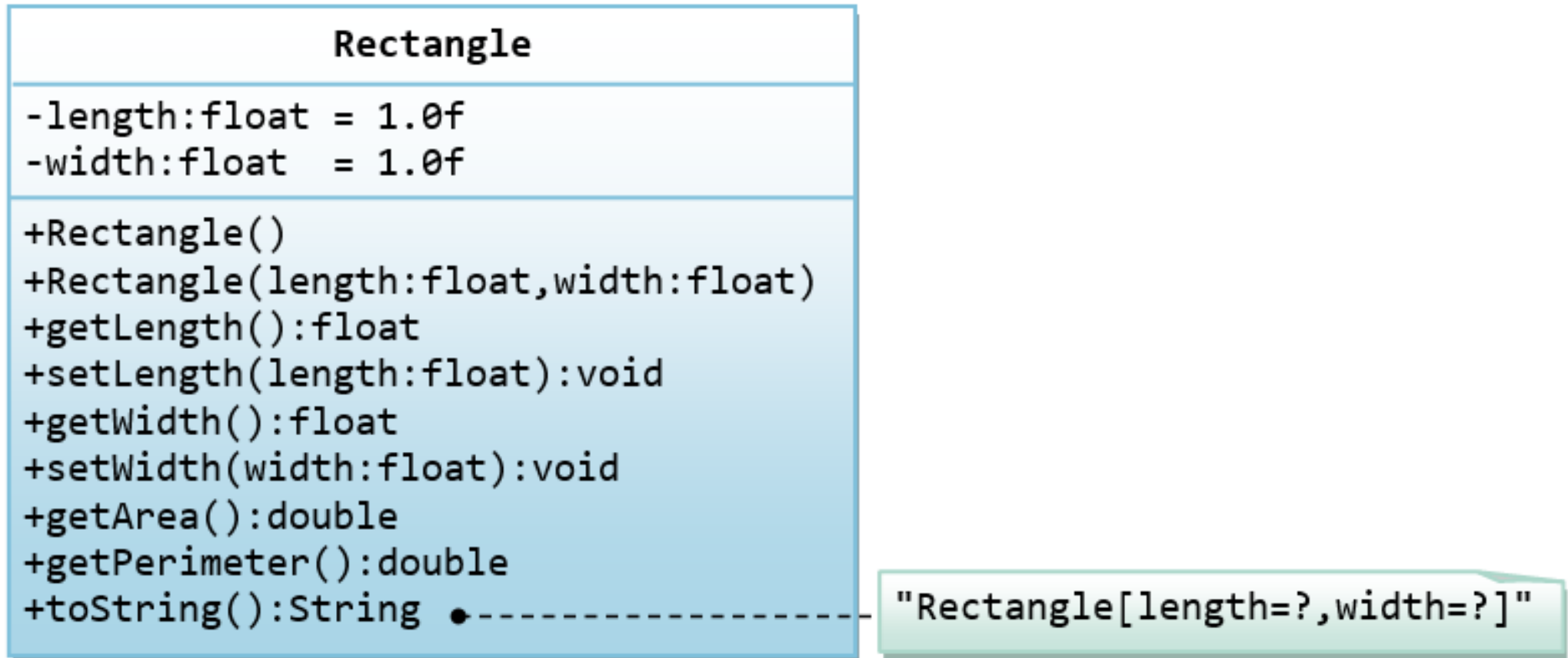
In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



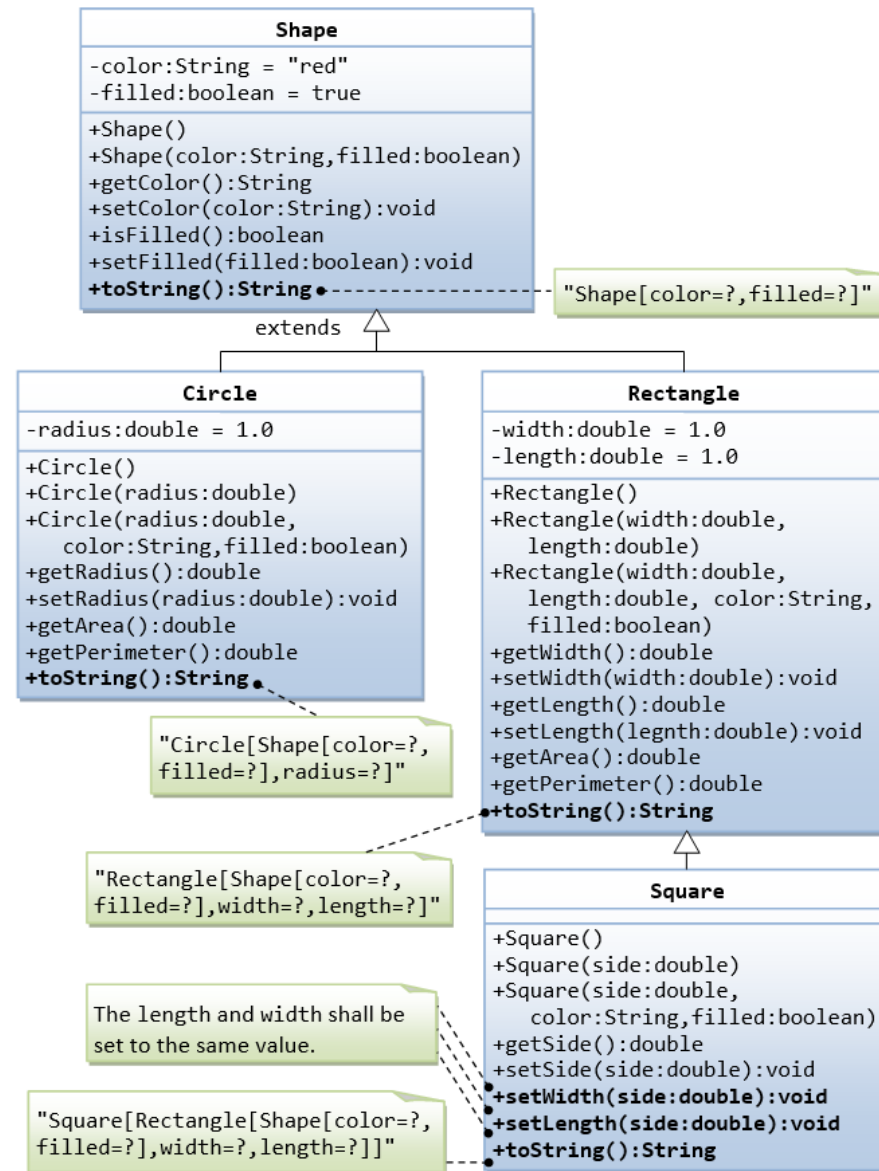
In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



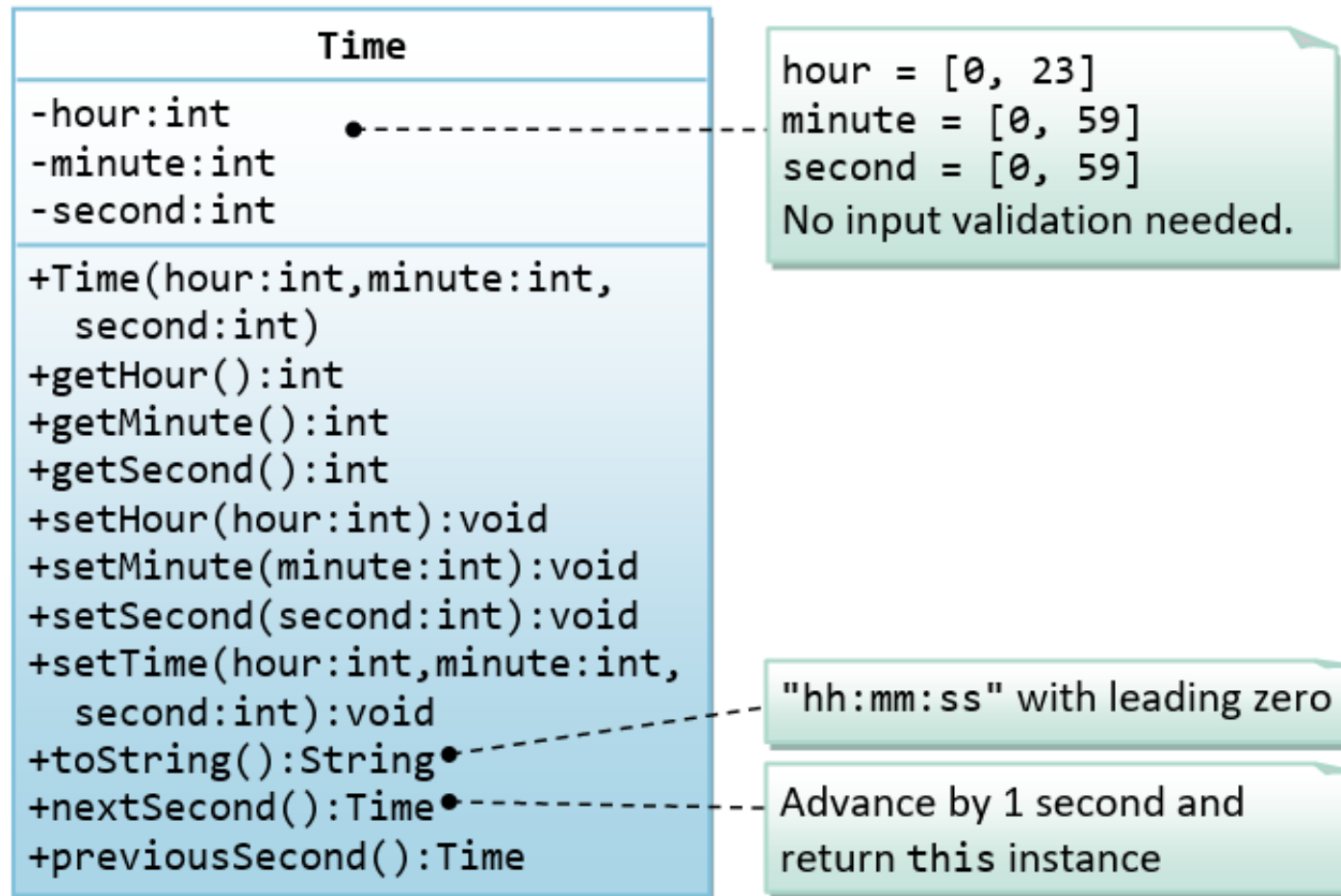
In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



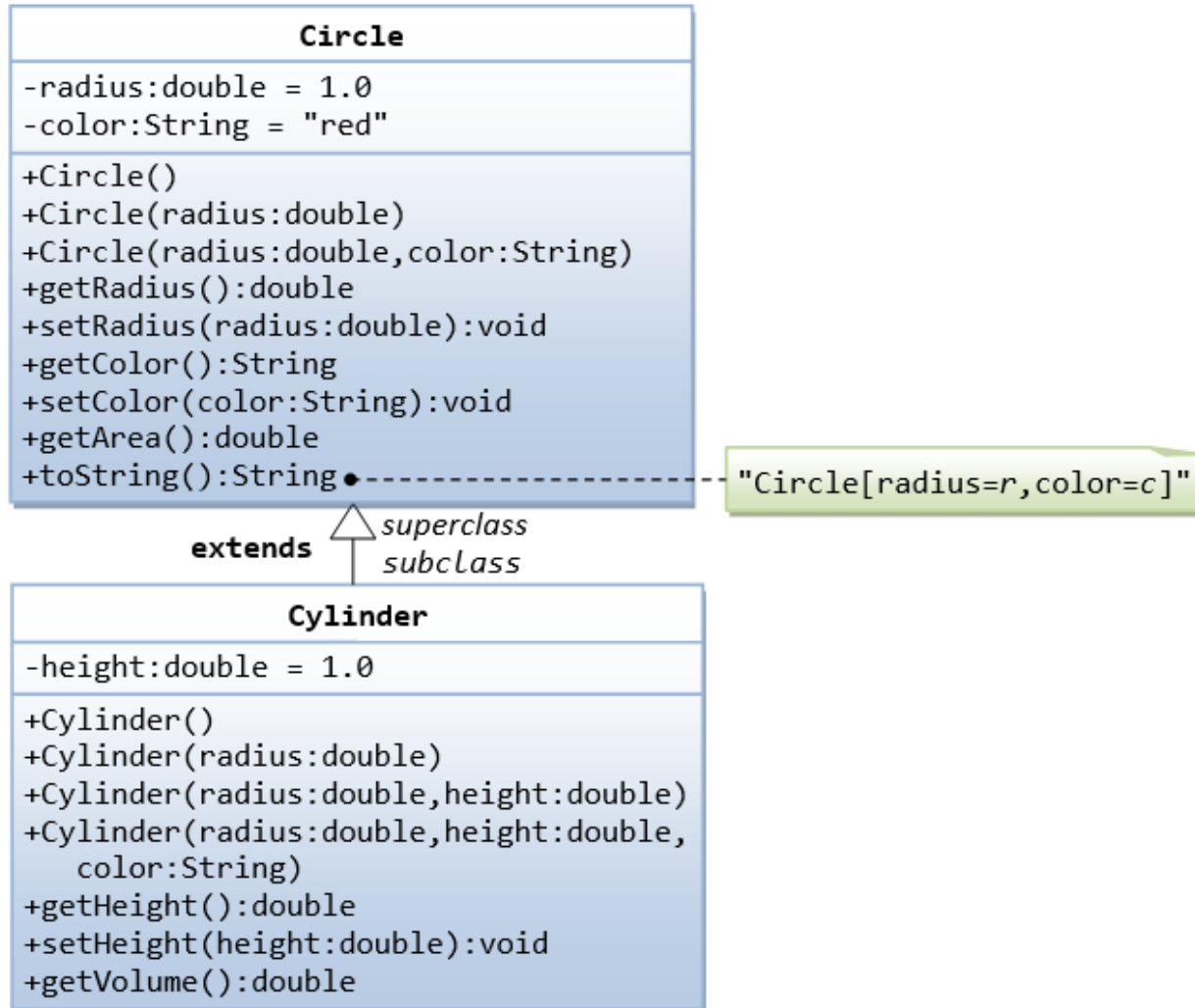
In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



In-class practice

- Implement following class diagram in Java. Also, write a test to use the implemented class.



Good luck 🍀



References:

<https://www.javatpoint.com/java-swing>

<https://www.tutorialspoint.com/swing>